

Ollama Embedding: How to Feed Data to AI for Better Response?



Mervin Praisson
75.2K subscribers

Join

Subscribe

2.3K



Share

Ask

Download



66,372 views Feb 23, 2024 #ollama #Chroma #Mistral

★ Welcome to our deep dive into Ollama Embedding for AI applications! In this comprehensive tutorial, we're unlocking the power of Ollama Embedding to enhance your app's performance significantly. ★ Feed Data to AI for better response.

Q What We Cover:

Introduction to Ollama Embedding and its advantages.

Step-by-step guide on ingesting data from URLs, converting them into embeddings, and storing in Vector Database using ChromaDB.

Integration with Nomic Embed Text for superior embedding performance.

Utilising RAG for data retrieval.

Building a user-friendly interface with Gradio.

If you like this video:

Tweet something positive or what you like about these tutorials on [Twitter](#) / @mervinpraisson

"@MervinPraisson"

Setup Steps:

Installation of necessary packages (Lang Chain, Chroma Embeddings, etc.)

Detailed walkthrough for setting up your application file.

Splitting data, converting to embeddings, and database storage.

Creating and integrating the user interface with Gradio.

0:00 - Introduction to Ollama Embedding

0:39 - Benefits of Nomic Embed Text

1:00 - User Interface Preview

1:04 - Subscription Reminder

1:21 - Setting Up Your Application

3:00 - Understanding the Rag Process

4:01 - Running the Code

5:00 - Adding User Interface with Gradio

Ollama Embedding



1 Data from URL

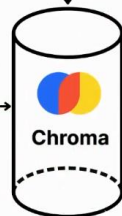
Ollama
Embedding

NOMIC



2

Question

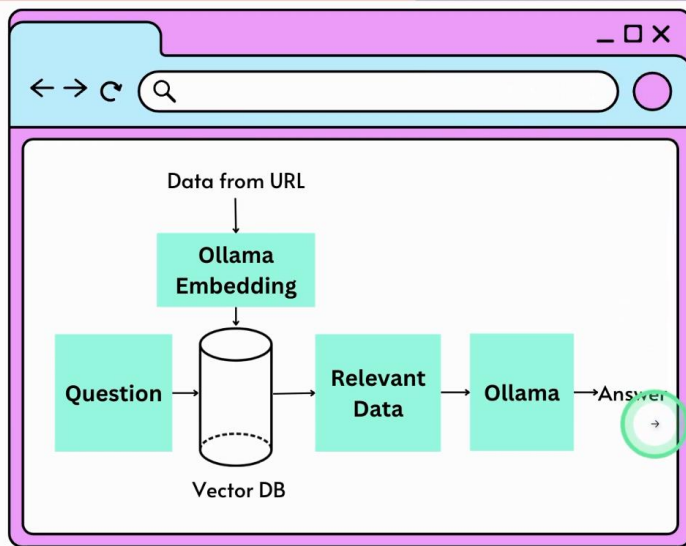


Vector DB

Relevant
Data

Ollama

Answer



nomic-embed-text

```
ollama run nomic-embed-text
```

A high-performing open embedding model with a 8192 token context window.

327 Pulls Updated yesterday

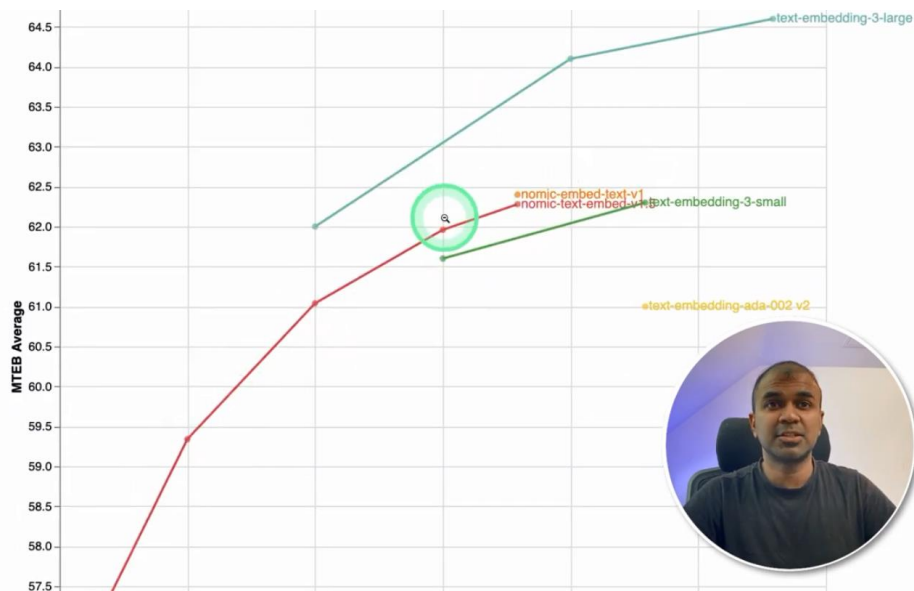


be used to generate embeddings.

nomic-embed-text is a 8192 context length text encoder that surpasses OpenAI text-embedding-ada-002 and text-embedding-3-small performance on short and long context tasks.

Usage

This model is an embedding model, meaning it can only be embeddings.



Document Query with Ollama

Enter URLs and a question to query the documents.

Enter URLs separated by new lines

<https://ollama.com/blog/>

Question

What is Ollama

Clear

Submit

output

Based on the context provided in the documents, Ollama is a platform that offers large language models which can be run and deployed locally, with features including access to a wide range of models, built-in GPU acceleration, support for Python and JavaScript, and the ability to run models on a variety of hardware. Ollama also includes features such as chat completion, code generation, and more. Ollama preview, can be used to run different versions of models, such as Llama 3.1 and Code Llama.

```
~/mer.vin > pip install langchain langchain-community langchain-core
```

```
~/mer.vin > touch app.py
```

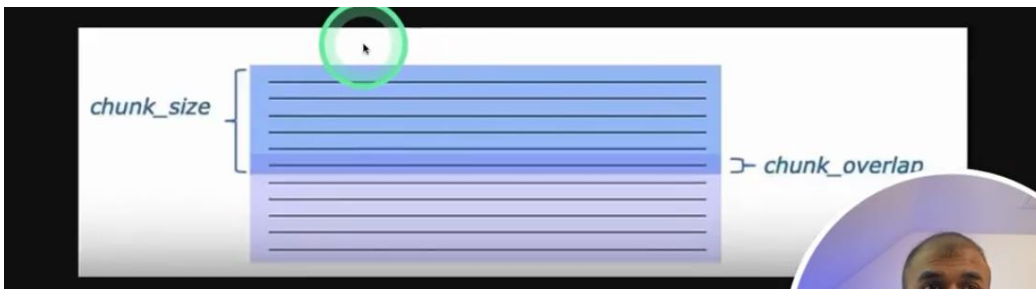
app.py

```
1 from langchain_community.document_loaders import WebBaseLoader
2 from langchain_community.vectorstores import Chroma
3 from langchain_community import embeddings
4 from langchain_community.chat_models import ChatOllama
5 from langchain_core.runnables import RunnablePassthrough
6 from langchain_core.output_parsers import StrOutputParser
7 from langchain_core.prompts import ChatPromptTemplate
8 from langchain.text_splitter import CharacterTextSplitter
9
10 model_local = ChatOllama(model="mistral")
11
```

app.py

```
11
12 # 1. Split data into chunks
13 urls = [
14     "https://ollama.com/",
15     "https://ollama.com/blog/windows-preview",
16     "https://ollama.com/blog/openai-compatibility",
17 ]
18 docs = [WebBaseLoader(url).load() for url in urls]
19 docs_list = [item for sublist in docs for item in sublist]
20 text_splitter = CharacterTextSplitter.from_tiktoken_encoder(chunk_size=7500, chunk_overlap=100)
21 doc_splits = text_splitter.split_documents(docs_list)
22
23 # 2. Convert documents to Embeddings and store them
```

```
18 WebBaseLoader(url).load() for url in urls]
19 t = [item for sublist in docs for item in sublist]
20 itter = CharacterTextSplitter.from_tiktoken_encoder(chunk_size=7500, chunk_overlap=100)
```



```
app.py
18 docs = [WebBaseLoader(url).load() for url in urls]
19 docs_list = [item for sublist in docs for item in sublist]
20 text_splitter = CharacterTextSplitter.from_tiktoken_encoder(chunk_size=7500, chunk_overlap=
21 doc_splits = text_splitter.split_documents(docs_list)
22
23 # 2. Convert documents to Embeddings and store them
24 vectorstore = Chroma.from_documents(
25     documents=doc_splits,
26     collection_name="rag-chroma",
27     embedding=embeddings.ollama.OllamaEmbeddings(model='nomic-embed-text'),
28 )
29 retriever = vectorstore.as_retriever()
30
31 # 3. Before RAG
32 print("Before RAG\n")
33 before_rag_template = "What is {topic}"
```

```
app.py
34 retriever = vectorstore.as_retriever()
35
36 # 3. Before RAG
37 print("Before RAG\n")
38 before_rag_template = "What is {topic}"
39 before_rag_prompt = ChatPromptTemplate.from_template(before_rag_template)
40 before_rag_chain = before_rag_prompt | model_local | StrOutputParser()
41 print(before_rag_chain.invoke({"topic": "Ollama"}))
42
43 # 4. After RAG
44 print("\n#####\nAfter RAG\n")
45 after_rag_template = ""Answer the question based only on the following context:
46 {context}
47 Question: {question}
48 ""
```

```
app.py
49 # 4. After RAG
50 print("\n#####\nAfter RAG\n")
51 after_rag_template = ""Answer the question based only on the following context:
52 {context}
53 Question: {question}
54 ""
55 after_rag_prompt = ChatPromptTemplate.from_template(after_rag_template)
56 after_rag_chain = (
57     {"context": retriever, "question": RunnablePassthrough()}
58     | after_rag_prompt
59     | model_local
60     | StrOutputParser()
61 )
62 print(after_rag_chain.invoke("What is Ollama?"))
```

```

app.py
7 from langchain_core.prompts import ChatPromptTemplate
8 from langchain.text_splitter import CharacterTextSplitter
9
10 model_local = ChatOllama(model="mistral")
11
12 # 1. Split data into chunks
13 urls = [
14     "https://ollama.com/",
15     "https://ollama.com/blog/windows-preview",
16     "https://ollama.com/blog/openai-compatibility",
17 ]

```

```

app.py
15     "https://ollama.com/blog/windows-preview",
16     "https://ollama.com/blog/openai-compatibility",
17 ]
18 docs = [WebBaseLoader(url).load() for url in urls]
19 docs_list = [item for sublist in docs for item in sublist]
20 text_splitter = CharacterTextSplitter.from_tiktoken_encoder(chunk_size=7500, chunk_overlap=
21 doc_splits = text_splitter.split_documents(docs_list)
22
23 # 2. Convert documents to embeddings and store them
24 vectorstore = Chroma.from_documents(
25     documents=doc_splits,
26     collection_name="rag-chroma",
27     embedding=embeddings.ollama.OllamaEmbeddings(model='nomic
28 )
29 retriever = vectorstore.as_retriever()
30
31 # 3. Before RAG
32 print("Before RAG\n")
33 before_rag_template = "What is {topic}"
34 before_rag_prompt = ChatPromptTemplate.from_template(before_rag_template)
35 before_rag_chain = before_rag_prompt | model_local | StrOutputParser()

```

```

app.py
34 before_rag_prompt = ChatPromptTemplate.from_template(before_rag_template)
35 before_rag_chain = before_rag_prompt | model_local | StrOutputParser()
36 print(before_rag_chain.invoke({"topic": "Ollama"}))
37
38 # 4. After RAG
39 print("\n#####\nAfter RAG\n")
40 after_rag_template = """Answer the question based only on the following context:
41 {context}
42 Question: {question}
43 """
44 after_rag_prompt = ChatPromptTemplate.from_template(after_rag_template)
45 after_rag_chain = (
46     {"context": retriever, "question": RunnablePassthrough()}
47     | after_rag_prompt
48     | model_local
49     | StrOutputParser()
50 )
51 print(after_rag_chain.invoke("What is Ollama?"))

```

```
> ollama pull nomic-embed-text
pulling manifest
pulling 970aa74c0a90... 100% 274 MB
pulling c71d239df917... 100% 11 KB
pulling ce4a164fc046... 100% 17 B
pulling 31df23ea7daa... 100% 420 B
verifying sha256 digest
writing manifest
removing any unused layers
success
> ollama pull mistral
pulling manifest
pulling e8a35b5937a5... 100% 4.1 GB
pulling 43070e2d4e53... 100% 11 KB
pulling e6836092461f... 100% 42 B
pulling ed11eda7790d... 100% 30 B
pulling f9b1e3196ecf... 100% 483 B
verifying sha256 digest ::
```



```
~/mer.vin > python app.py
```

```
> python app.py
Before RAG
```

I'm sorry for the confusion, but "Ollama" doesn't seem to be a widely recognized term or acronym in various fields such as technology, science, mathematics, or finance. It may refer to a specific term in a particular context or industry that is not known to me. If you could please provide some additional context or information about where you encountered this term, I'd be happy to help you out with any related queries you might have!

```
#####
After RAG
```

Number of requested results 4 is greater than number of elements in index

```
time=2024-02-23T17:04:44.962Z level=
INFO source=dyn_ext_server.go:161 ms
g="Starting llama main loop"
[GIN] 2024/02/23 - 17:04:44 | 200 |
219.98125ms | 127.0.0.1 | POST
"/api/embeddings"
time=2024-02-23T17:04:44.982Z level=
INFO source=routes.go:78 msg="changing loaded model"
ggml_metal_free: deallocating
time=2024-02-23T17:04:44.982Z level=
INFO source=dyn_ext_server.go:161 ms
g="Loading Dynamic
olders/0b/t9c4hwy
00gn/T/ollama255
_server.dylib"
time=2024-02-23T17:04:44.982Z level=
INFO source=dyn_ext_server.go:161 ms
g="Initializing llama
llama_model_loader: loading data
with 24 key-value pairs and 291 ten
```



related queries you might have!

```
#####
After RAG
```

Number of requested results 4 is greater than number of elements in index 3, updating n_results = 3

Ollama is a local AI model server that allows users to run large language models on their own machines, providing an OpenAI API-compatible endpoint for easy integration with various tools and applications. It currently supports models like Llama2 and Mistral. To use it, you first need to download Ollama and pull a desired model. Then, you can interact with Ollama using cURL or libraries such as OpenAI's Python and JavaScript SDKs. Additionally, it offers compatibility with various

```
time=2024-02-23T17:04:44.962Z level=
INFO source=dyn_ext_server.go:161 ms
g="Starting llama main loop"
[GIN] 2024/02/23 - 17:04:44 | 200 |
219.98125ms | 127.0.0.1 | POST
"/api/embeddings"
time=2024-02-23T17:04:44.982Z level=
INFO source=routes.go:78 msg="changing loaded model"
ggml_metal_free: deallocating
time=2024-02-23T17:04:44.982Z level=
INFO source=dyn_ext_server.go:161 ms
g="Loading Dynamic
olders/0b/t9c4hwy
00gn/T/ollama255
_server.dylib"
time=2024-02-23T17:04:44.982Z level=
INFO source=dyn_ext_server.go:161 ms
g="Initializing llama
llama_model_loader: loading data
with 24 key-value pairs and 291 ten
```



ter than number of elements in index 3, updating n_results = 3

Ollama is a local AI model server that allows users to run large language models on their own machines, providing an OpenAI API-compatible endpoint for easy integration with various tools and applications. It currently supports models like Llama2 and Mistral. To use it, you first need to download Ollama and pull a desired model. Then, you can interact with Ollama using cURL or libraries such as OpenAI's Python and JavaScript SDKs. Additionally, it offers compatibility with various frameworks like Vercel AI SDK and Autogen for building conversational applications. Future improvements include embeddings API, function calling, vision support, and logprobs.

~/mer.vin > █

```
time=2024-02-23T17:04:44.962Z level=
INFO source=dyn_ext_server.go:161 ms
g="Starting llama main loop"
[GIN] 2024/02/23 - 17:04:44 | 200 |
219.98125ms | 127.0.0.1 | PO
ST "/api/embeddings"
time=2024-02-23T17:04:44.982Z level=
INFO source=routes.go:78 msg="changi
ng loaded model"
ggml_metal_free: deallocating
time=2024-02-23T17:04:44.982Z level=
INFO source=dyn_ext_server.go:161 ms
g="Loading Dynamic
olders/0b/t9c4hwy
00gn/T/ollama255
_server.dylib"
time=2024-02-23T17:04:44.982Z level=
INFO source=dyn_ext_server.go:161 ms
g="Initializing llama
llama_model_loader: loading data
with 24 key-value pairs and 291 ten
```

We now have RAG app that runs completely locally with zero cost.

```
ui.py
1 import gradio as gr
2 from langchain_community.document_loaders import WebBaseLoader, PyPDFLoader
3 from langchain_community.vectorstores import Chroma
4 from langchain_community import embeddings
5 from langchain_community.chat_models import ChatOllama
6 from langchain_core.runnables import RunnablePassthrough
7 from langchain_core.output_parsers import StrOutputParser
8 from langchain_core.prompts import ChatPromptTemplate
9 from langchain.output_parsers import PydanticOutputParser
10 from langchain.text_splitter import CharacterTextSplitter
11
```

```
ui.py
8 from langchain_core.prompts import ChatPromptTemplate
9 from langchain.output_parsers import PydanticOutputParser
10 from langchain.text_splitter import CharacterTextSplitter
11
12 def process_input(urls, question):
13     model_local = ChatOllama(model="mistral")
14
```

```
ui.py
12 def process_input(urls, question):
13     model_local = ChatOllama(model="mistral")
14
15     # Convert string of URLs to list
16     urls_list = urls.split(",")
17     docs = [WebBaseLoader(url).load() for url in urls_list]
18     docs_list = [item for sublist in docs for item in sublist]
19
20     text_splitter = CharacterTextSplitter.from_tiktoken_encoder(chunk_size=1000)
21     doc_splits = text_splitter.split_documents(docs_list)
22
23     vectorstore = Chroma.from_documents(doc_splits,
```

```

23 vectorstore = Chroma.from_documents(
24     documents=doc_splits,
25     collection_name="rag-chroma",
26     embedding=embeddings.ollama.OllamaEmbeddings(model='nomic-embed-t
27 )
28 retriever = vectorstore.as_retriever()
29
30 after_rag_template = """Answer the question based only on the followi
31 {context}
32 Question: {question}
33 """
34 after_rag_prompt = ChatPromptTemplate.from_t
35 after_rag_chain = (
36     {"context": retriever, "question": Runnable
37     | after_rag_prompt

```

```

37     | after_rag_prompt
38     | model_local
39     | StrOutputParser()
40 )
41 return after_rag_chain.invoke(question)
42
43 # Define Gradio interface
44 iface = gr.Interface(fn=process_input,
45                     inputs=[gr.Textbox(label="Enter URLs separated by new lines",
46                                         outputs="text",
47                                         title="Document Query with Ollama",
48                                         description="Enter URLs and a question to query the documents.")],
49                     iface.launch()
50

```

```

45     label="Enter URLs separated by new lines"), gr.Textbox(label="Question")],
46
47     with Ollama",
48     s and a question to query the documents.")
49

```

```

> python ui.py
Running on local URL: http://127.0.0.1:7860

To create a public link, set `share=True` in `launch()`.

```

Document Query with Ollama

Enter URLs and a question to query the documents.

Enter URLs separated by new lines

Question

Clear

Submit

output



Use via API  · Built with Gradio 

Document Query with Ollama

Enter URLs and a question to query the documents.

Enter URLs separated by new lines

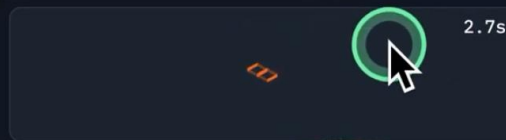
<https://ollama.com/blog/>
<https://ollama.com/>

Question

What is Ollama?

Clear

Submit



Use via API  · Built with Gradio 

Enter URLs separated by new lines

<https://ollama.com/blog/>
<https://ollama.com/>

Question

What is Ollama?

Clear

Submit

output

Based on the context provided, Ollama is a platform or software that enables users to run and create large language models locally, with support for macOS, Linux, and Windows (preview). It offers various features such as built-in GPU acceleration, access to the full model library, OpenAI compatibility, Python and JavaScript libraries, and the ability to run models like Llama 2, Code Llama, and vision models. Additional features include documentation on how to use them for specific purposes, problems or chat applications, as an official Docker container, and integration with various tools like GitHub, and note-taking applications.



Flag